Cube配置
- 1. 基本配置(Systick+OLED)
  - 1-1 Cube配置
  - 1-2 基础代码
- 2. GPIO配置
  - 2-1 Cube配置
  - 2-2 基础代码
- 3. TIM中断定时
  - 3-0 **中断必看**
  - 3-1 Cube配置(TIM3)
  - 3-2 基础代码
- 4. TIM外部中断
  - 4-1 Cube配置
  - 4-2 基础代码
- 5. TIM输出比较(PWM)
  - 5-1 Cube配置(TIM3)
  - 5-2 基础代码
- 6. TIM输入捕获
  - 6-1 Cube配置(TIM5)
  - 6-2 基础代码
- 7. TIM编码器模式
  - 7-1 Cube配置
  - 7-2 基础代码
- 8. USART串口
  - 8-1 Cube配置
  - 8-2 基础代码
- 9. USART_DMA
  - 9-1 Cube配置
  - 9-2 基础代码

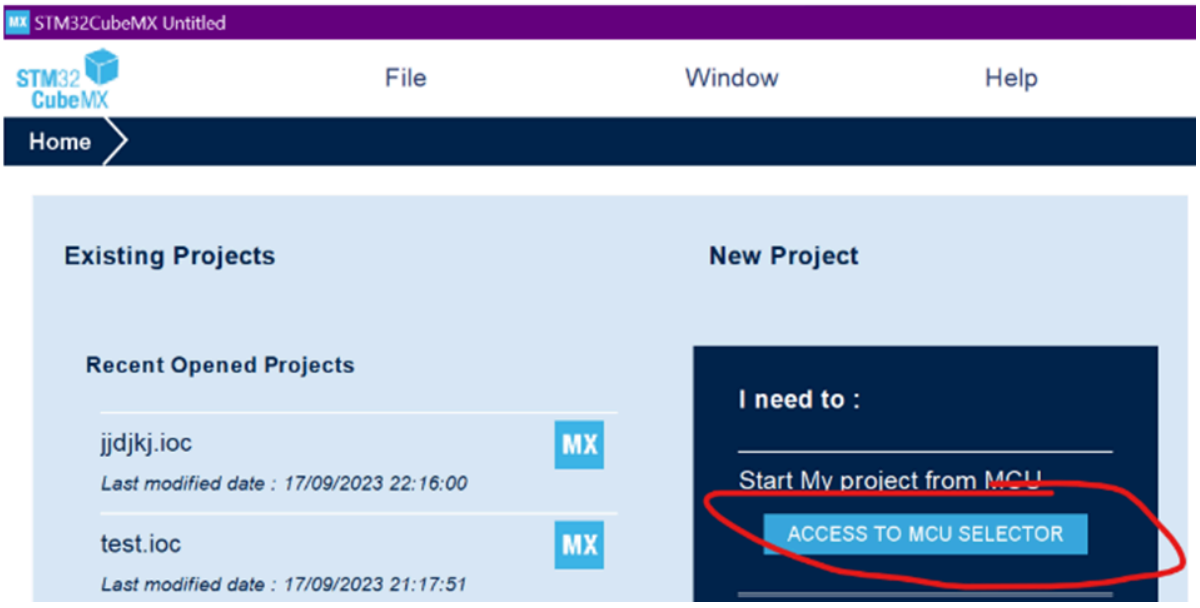
Cube配置

1. 基本配置(Systick+OLED)

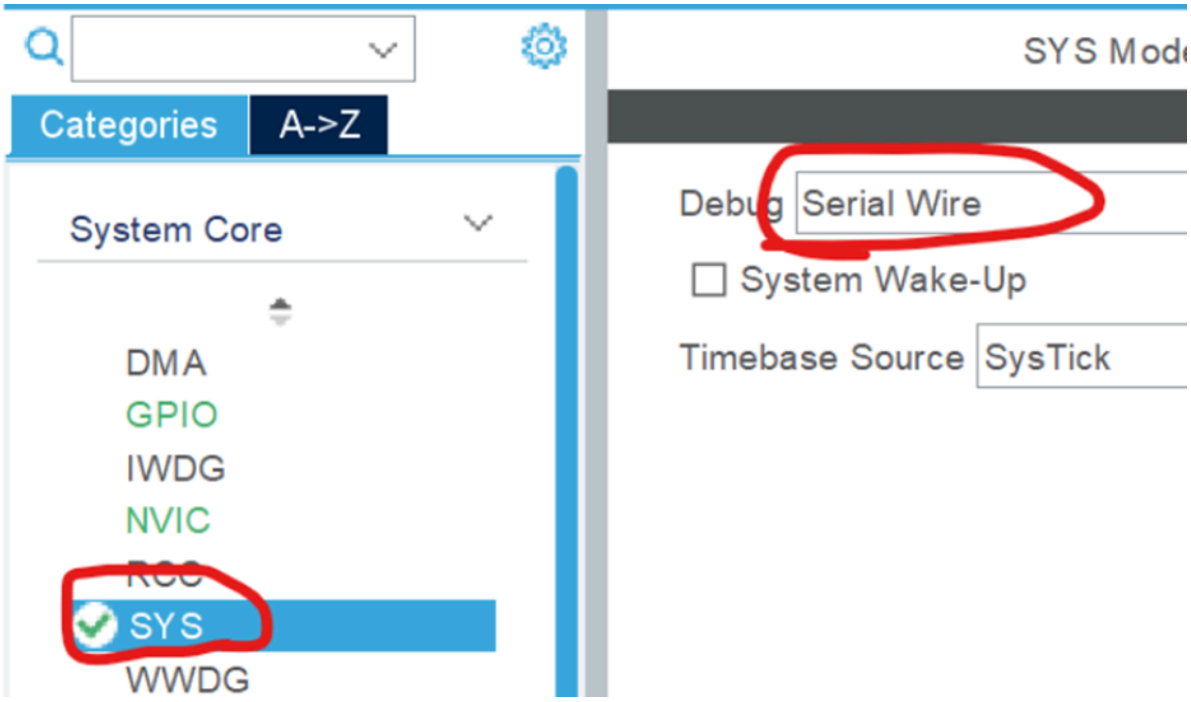
1-1 Cube配置

1. 进入页面(是**MCU**)

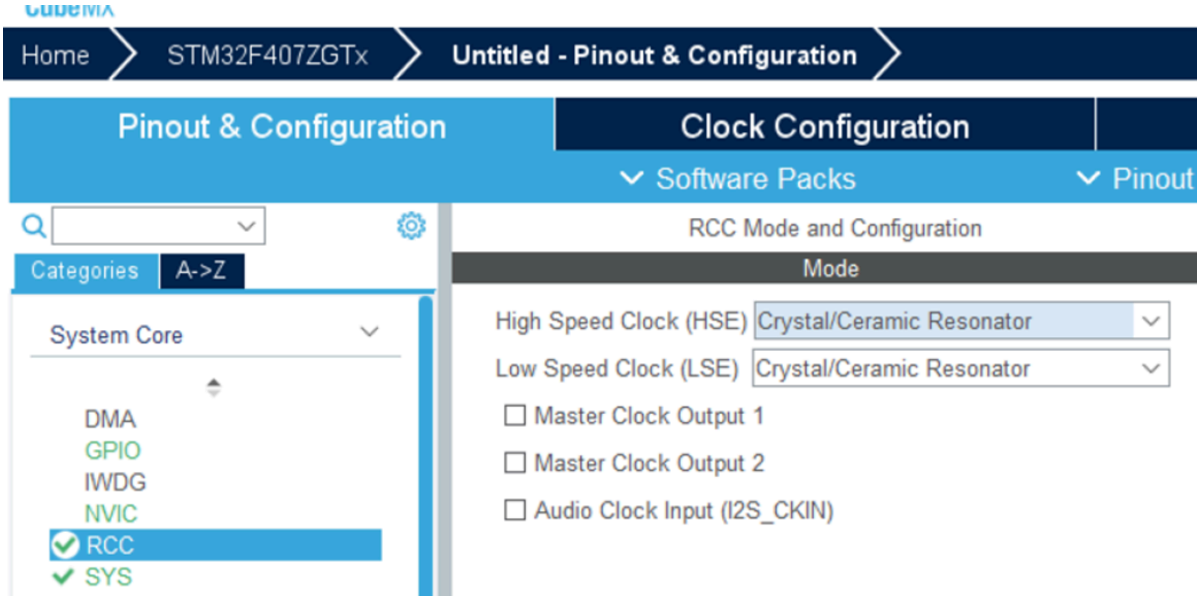
点击ACCESS TO MCU SELECTOR。



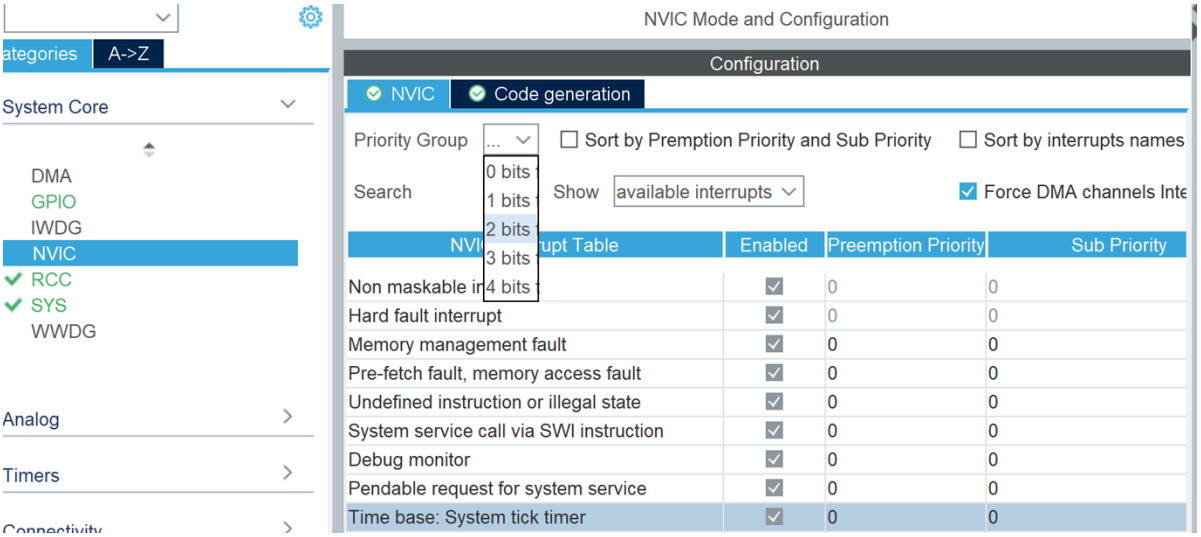
2. SYS配置



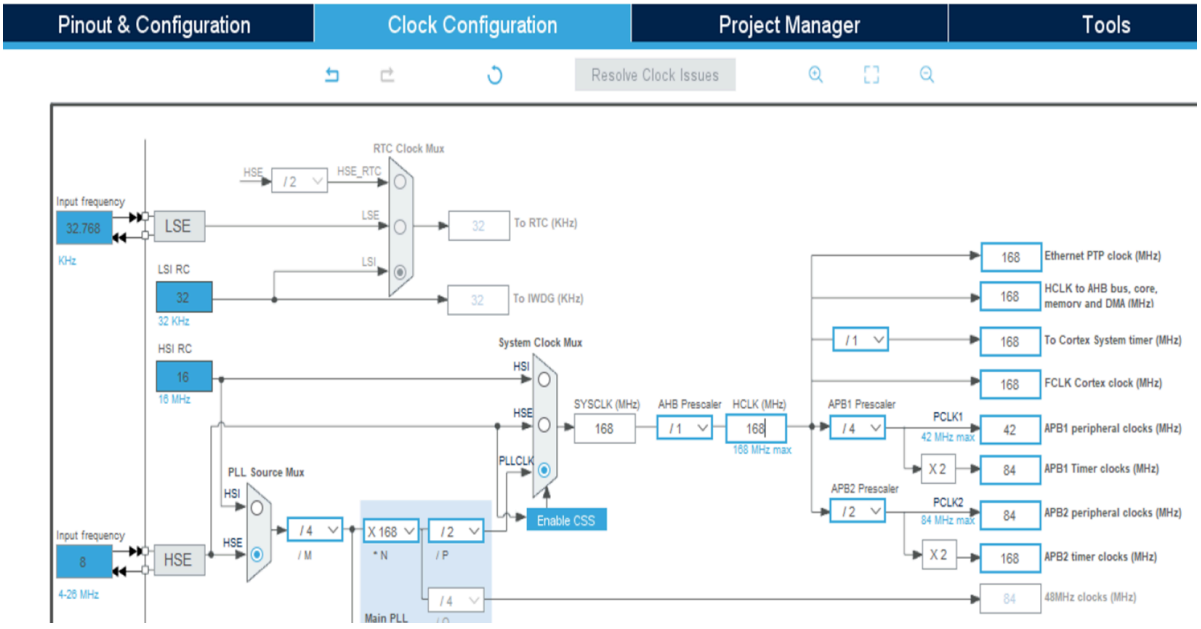
3. RCC配置



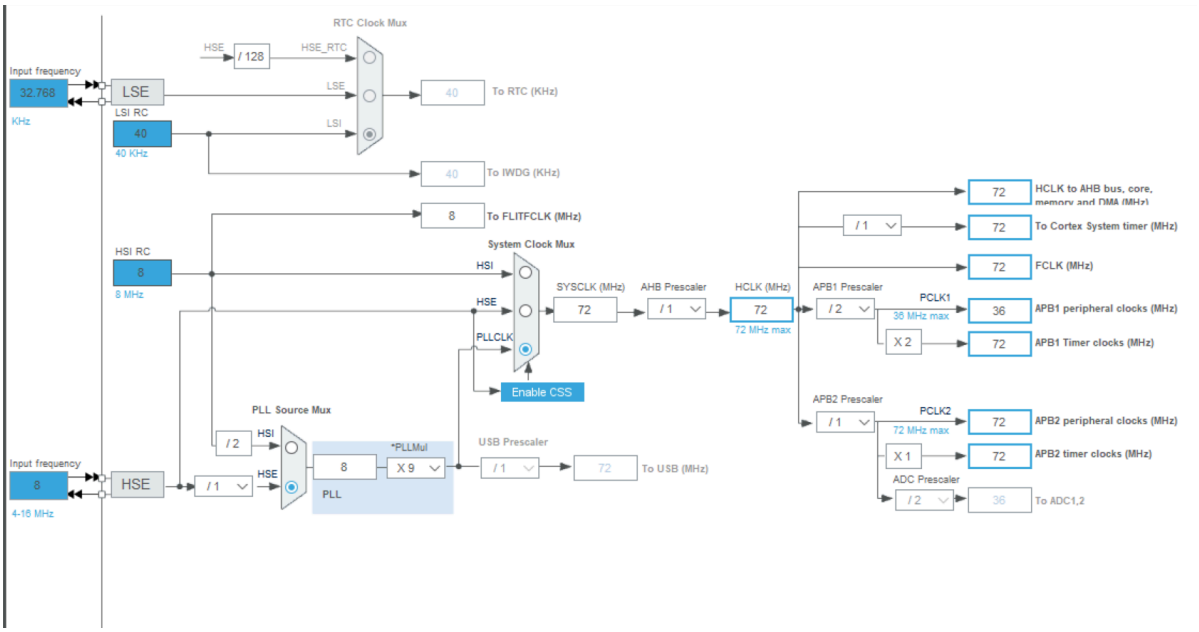
4. NVIC配置



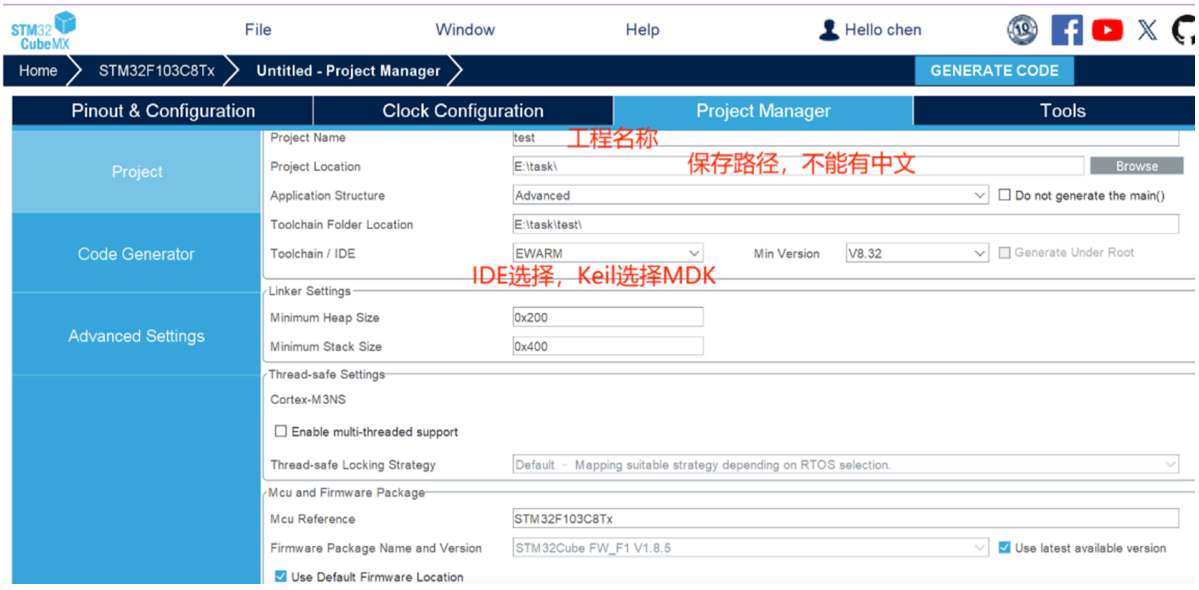
5. 时钟配置



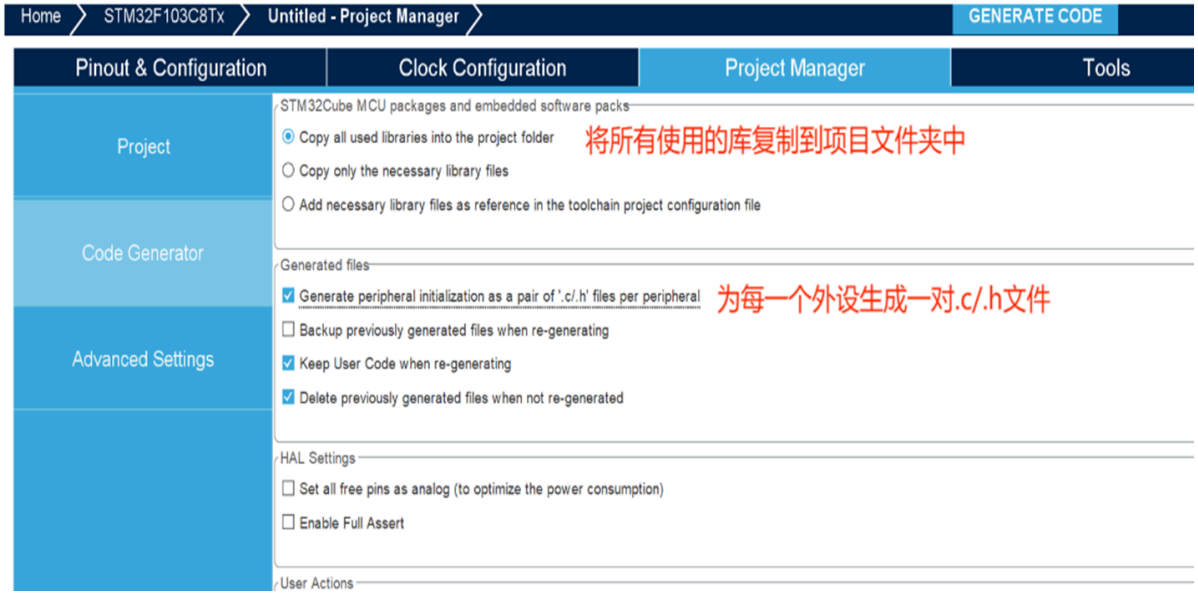
注:STM32F103配置:



6. 工程配置---1

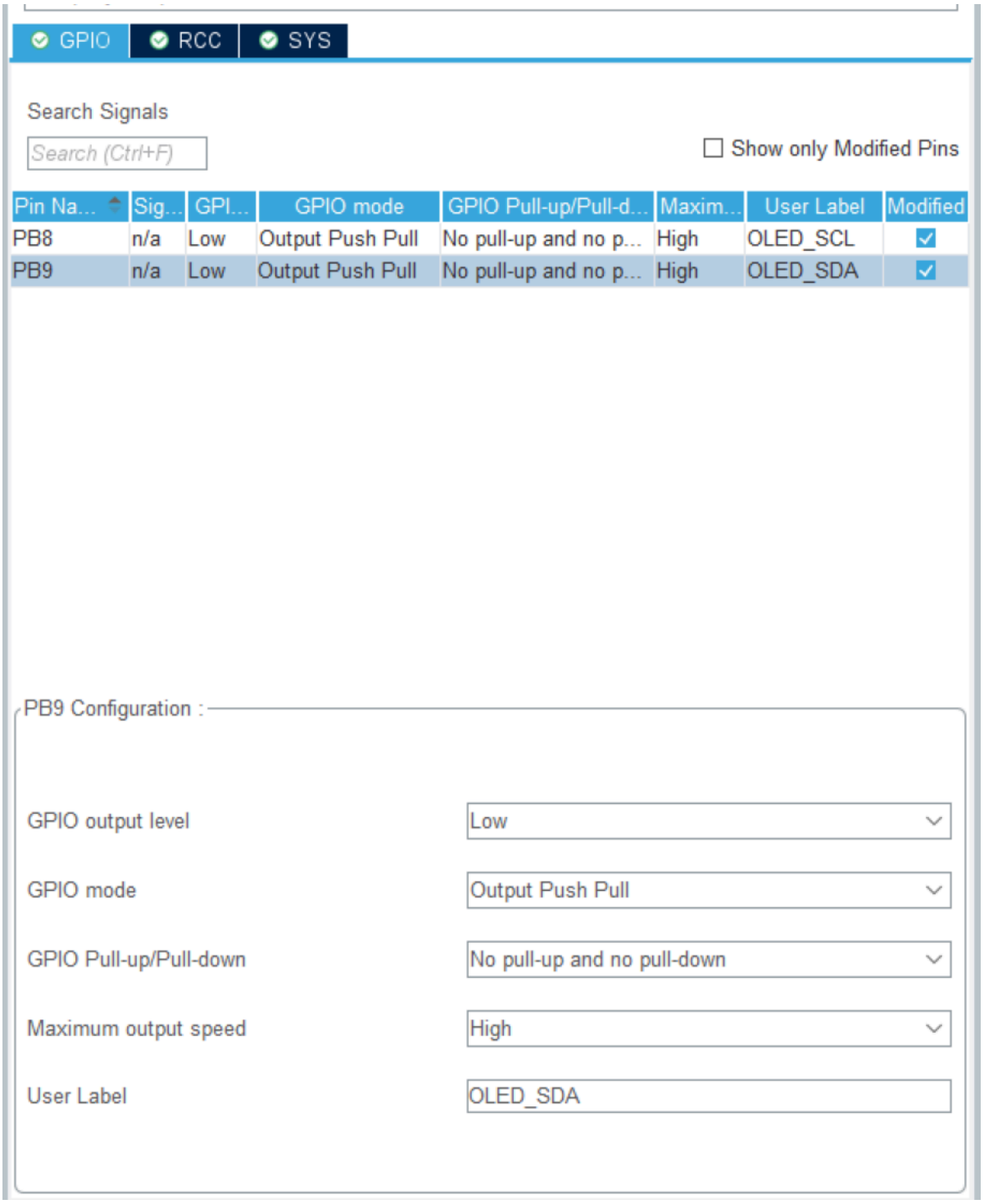


7. 工程配置---2



8. OLED引脚配置

注:STM32F1与F4都是这个配置



1-2 基础代码

1. 前提

```
主函数声明core_cm4.h
main.h添加:void HAL_SYSTICK_Callback(void) ;
stm32f4xx_it.c :找到函数SysTick_Handler,添加:HAL_SYSTICK_Callback() ;
```

2. setup

```
HAL_SYSTICK_Config(SystemCoreClock / 1000); //0.001s定时周期(SystemCoreClock / x)
```

3. 回调函数

```c
// 系统滴答定时器中断回调函数（每1ms调用一次）
void HAL_SYSTICK_Callback(void)
{
    static volatile uint32_t ms_count = 0;

    ms_count++; // 每毫秒递增计数器

    // 检查是否达到1000ms（1秒）
    if(ms_count >= 1000)
    {
        // 执行:LED状态翻转
        HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);

        ms_count = 0; // 重置计数器（避免溢出）
    }
}
```

4. OLED代码:

```c
// Setup:OLED初始化
OLED_Init() ;
OLED_ShowString(0,0,"Hello",OLED_8X16) ;
```

```c
// while:OLED刷新
// OLED刷新(必要,放在最后)
OLED_Update() ;
```

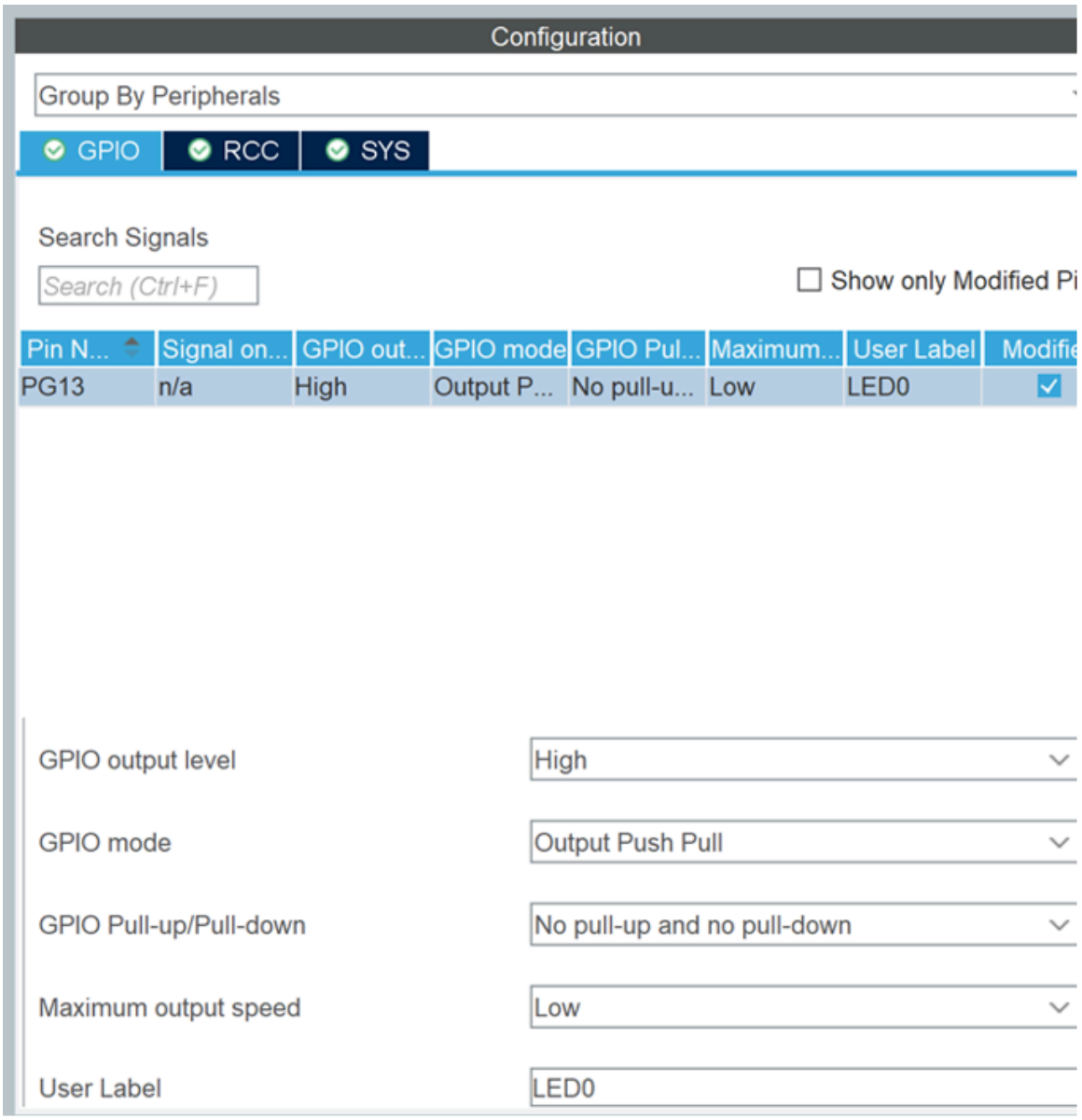
2. GPIO配置

2-1 Cube配置

1. 输出模式(OUTPUT)

   参数解释:

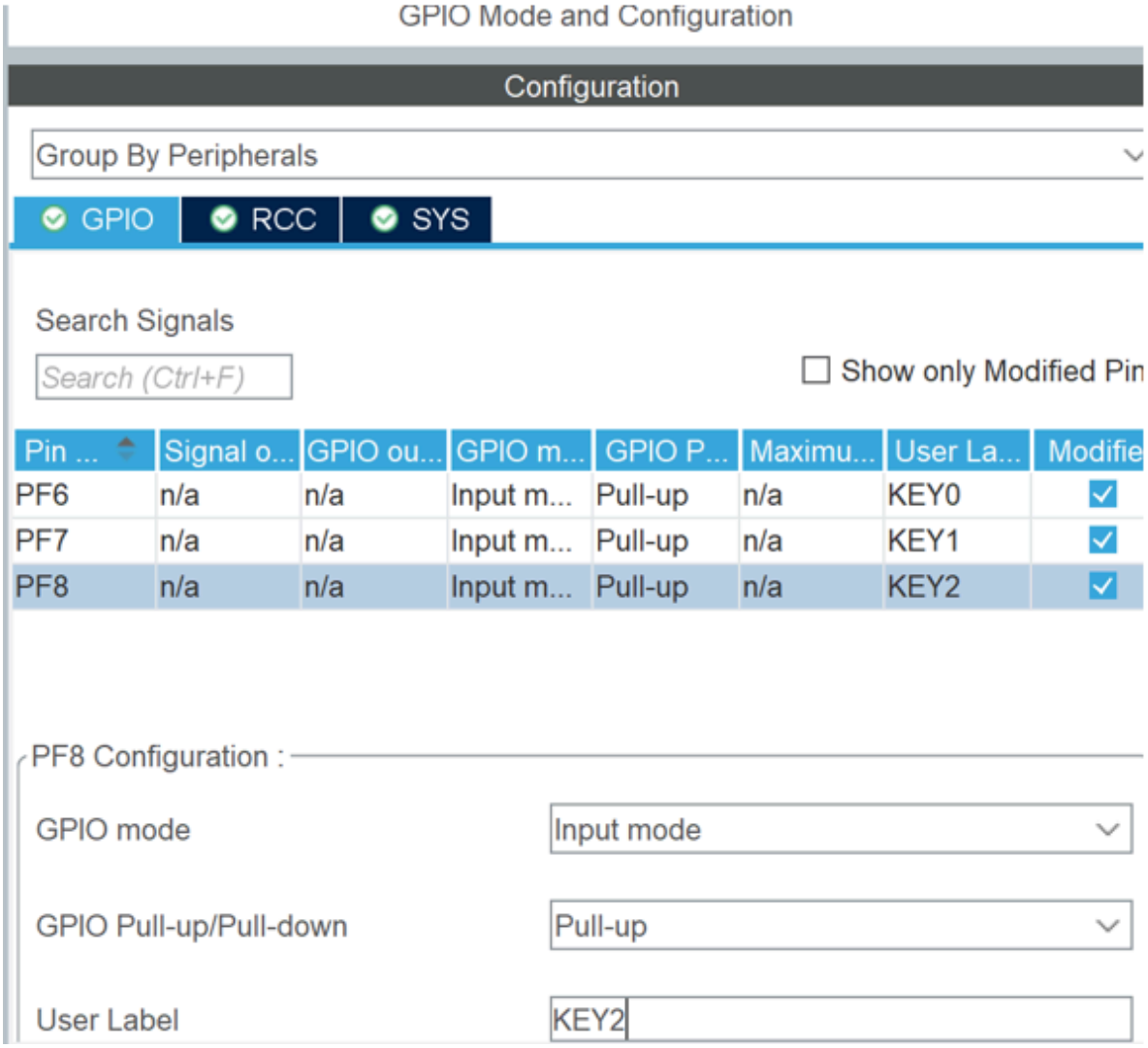
- GPIO output level : 引脚的初始状态(输出高电平or低电平),这里是给系统板载LED配置输出,根据原理图是引脚对VCC,所以初始为HIGH可以保证LED初始为暗
- GPIO mode : 推挽输出(PP)和开漏输出(OD)
- GPIO Pull-up / Pull-down : 选择上拉/下拉/浮空
- Maximun output speed : 输出速度,不了解,随便给呗
- User Label : 宏定义名字,建议给相关命名



2. 输入模式(Input)

参数解释:

见上面Output



2-2 基础代码

省略

3. TIM中断定时

注:对于中断,F4的84M,F1为72M,所以使用F1时将84改为72,代码就差不多完全一样了

3-0 中断必看

- STM32F4定时器功能分组

通用定时器包括TIM2~ TIM5， TIM9~TIM14，其中：
TIM2~ TIM5可以向上、向下和向上/向下计数， TIM9~TIM14只能向上计数。
TIM2~ TIM5支持编码器接口模式， TIM9~TIM14不支持。
TIM2~TIM5的捕获/比较通道有4路；TIM9和TIM12有2路；TIM10、TIM11 和 TIM13、TIM14只有1路。
TIM2~ TIM5 、 TIM9和TIM12的时钟选择有多种； TIM10、TIM11 和 TIM13、TIM14的计数器时钟只能选择内部时钟；

- STM32F4定时器时钟频率

1. 当APB1和APB2分频数为1的时候，TIM1、TIM8~TIM11的时钟为APB2的时钟，TIM2~TIM7、TIM12~TIM14的时钟为APB1的时钟；

2. 而如果APB1和APB2分频数不为1，那么TIM1、TIM8~TIM11的时钟为APB2的时钟的两倍，TIM2~TIM7、TIM12~TIM14的时钟为APB1的时钟的两倍。
3. 讲人话:由于在Cube中已经定义了时钟的分频(4),所以只看第2点就行了,**下面是定时器时钟频率简易版**:

   APB1（时钟 84 MHz）,这些定时器的 定时器时钟频率 = 84 MHz。

   TIM2 , TIM3 , TIM4 , TIM5 , TIM6 , TIM7 , TIM12 , TIM13 , TIM14

   APB2（时钟 168 MHz）,这些定时器的 定时器时钟频率 = 168 MHz。

   TIM1 （高级定时器）,TIM8 （高级定时器）, TIM9 , TIM10 , TIM11

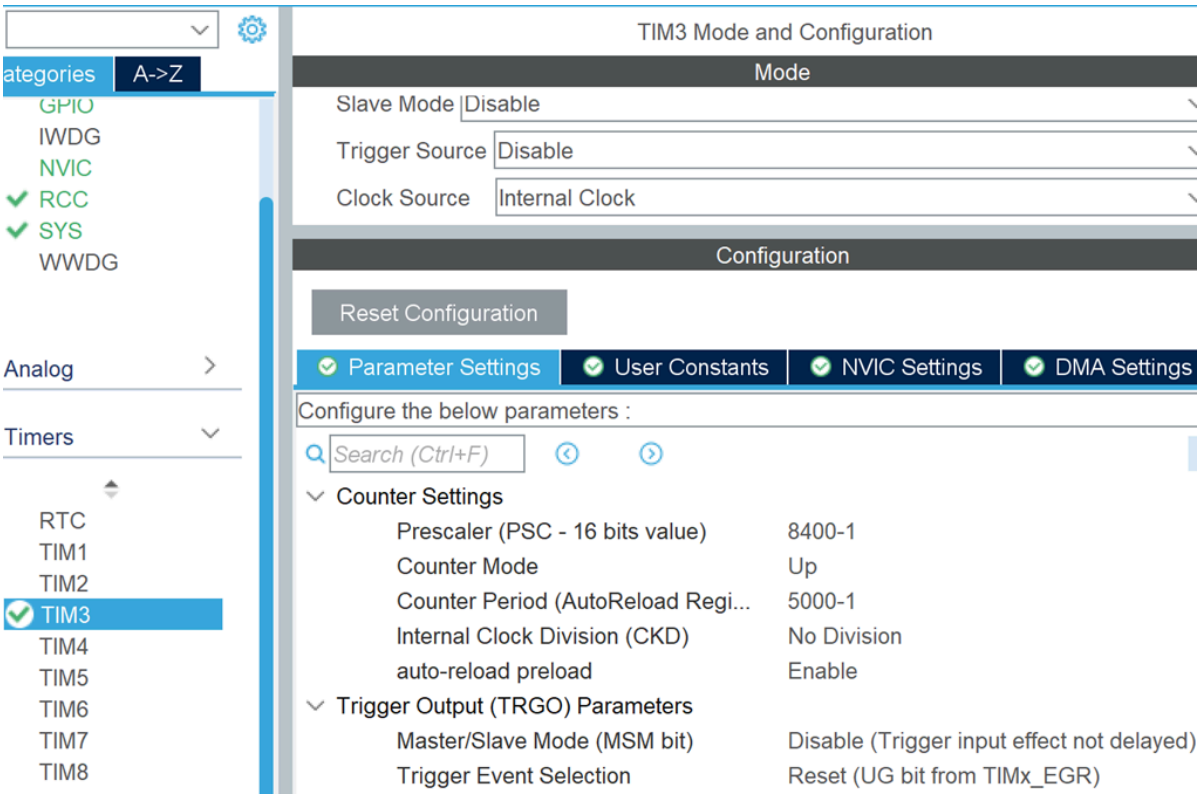
- STM32F1定时器功能分组

类型	编号	总线	功能
高级定时器	TIM1、TIM8	APB2	拥有通用定时器全部功能，并额外具有重复计数器、死区生成、互补输出、刹车输入等功能
通用定时器	TIM2、TIM3、TIM4、TIM5	APB1	拥有基本定时器全部功能，并额外具有内外时钟源选择、输入捕获、输出比较、编码器接口、主从触发模式等功能
基本定时器	TIM6、TIM7	APB1	拥有定时中断、主模式触发DAC的功能

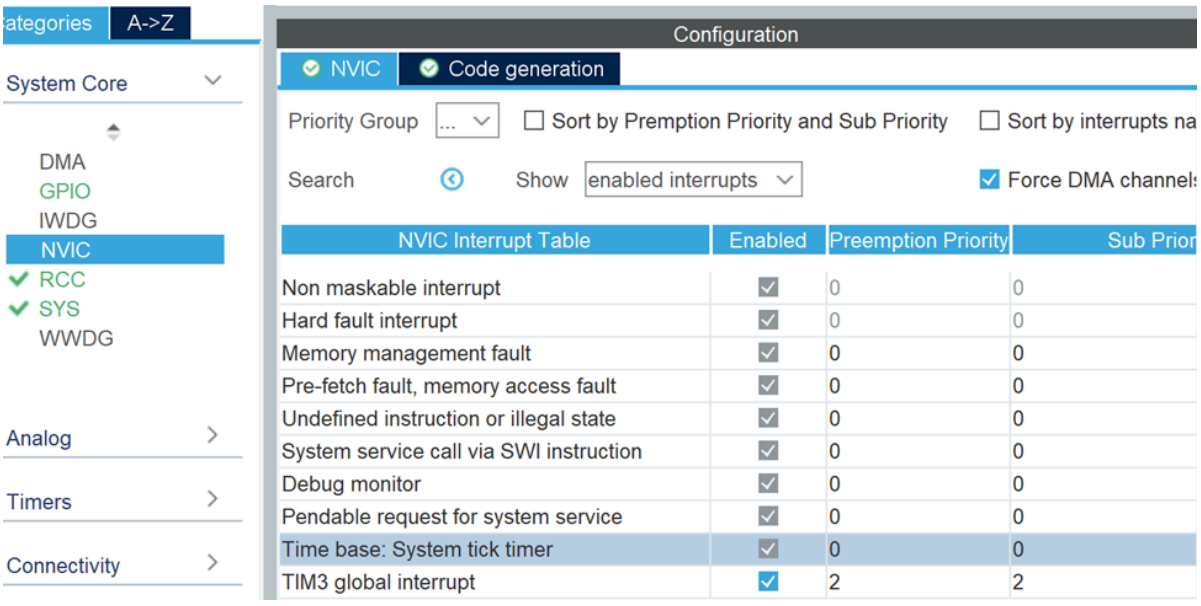
- STM32F103C8T6定时器资源：TIM1、TIM2、TIM3、TIM4
- STM32F1定时器时钟频率
- **都是72M**

3-1 Cube配置(TIM3)

1. 参数配置



2. 打开NVIC中断(重点,不然没法定时)



3-2 基础代码

1. 前提

```
// setup启动定时器中断模式
HAL_TIM_Base_Start_IT(&htim3);
```

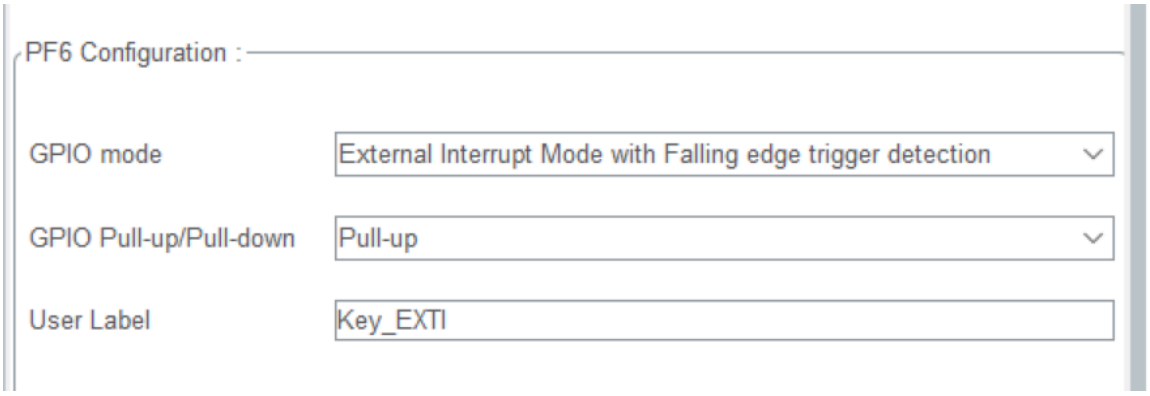
2. 核心代码

```c
// TIM3定时器
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  //判断是否是TIM3中断
  if (htim->Instance == TIM3)
  {
    // 执行功能:
    HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);
  }
}
```

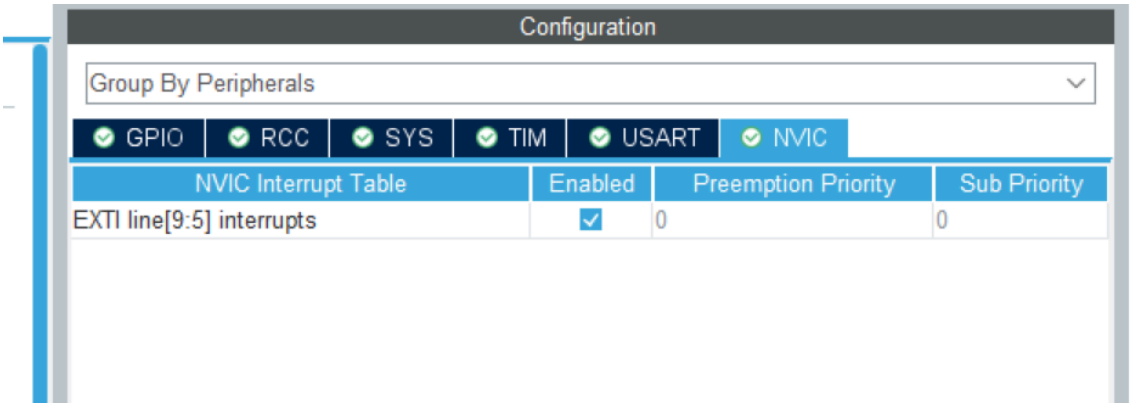
4. TIM外部中断

4-1 Cube配置

1. GPIO配置参数
   - mode:可以选择上升沿or下降沿or双触发(中断)
   - 上下拉自己选择,比如说这里就是选择上拉+下降沿触发+按键,所以按下按键触发中断
   - Label记得取一个名字



2. 不要忘了开启外部中断哦



4-2 基础代码

自己写回调函数

```c
// 自己写回调函数
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == Key_EXTI_Pin)   // 判断引脚号
    {
        // 执行功能
        check_1 ++ ;
//      __HAL_GPIO_EXTI_CLEAR_FLAG(Key_EXTI_Pin) ;  // 不需要重置标志位,HAL已经做好了
    }
}
```

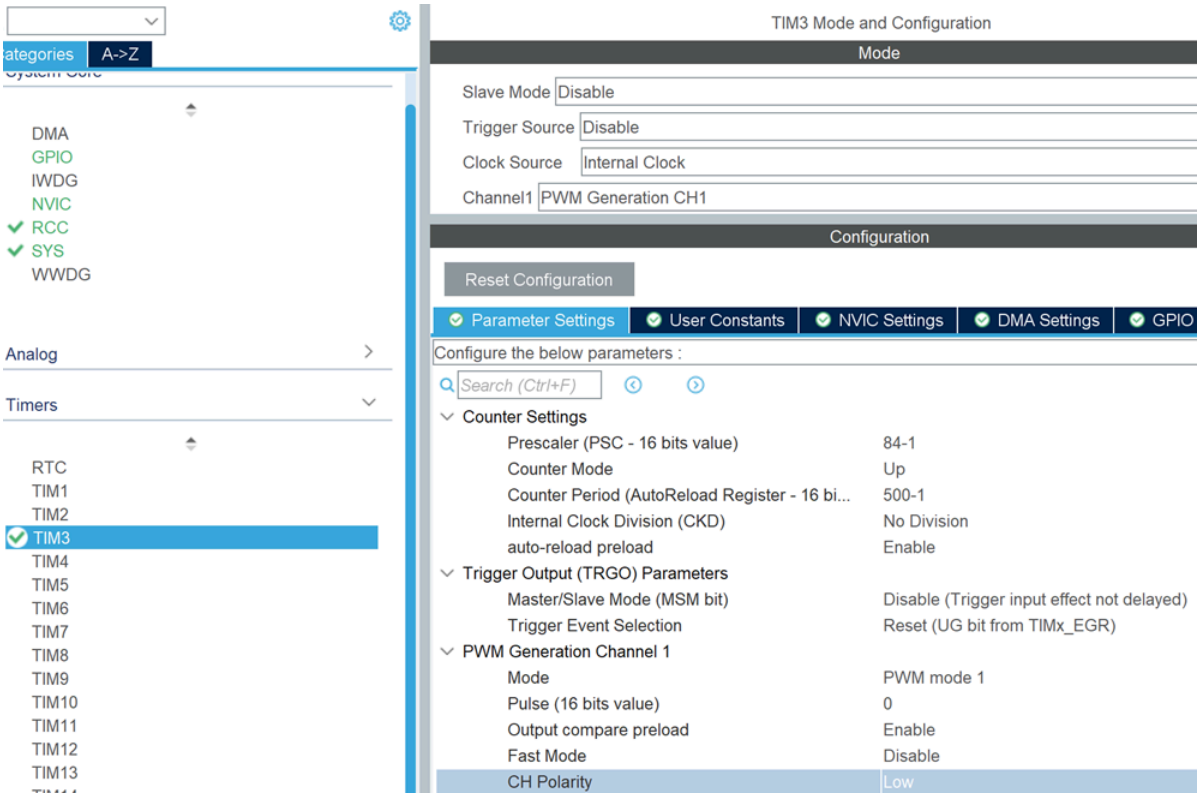
5. TIM输出比较(PWM)

5-1 Cube配置(TIM3)

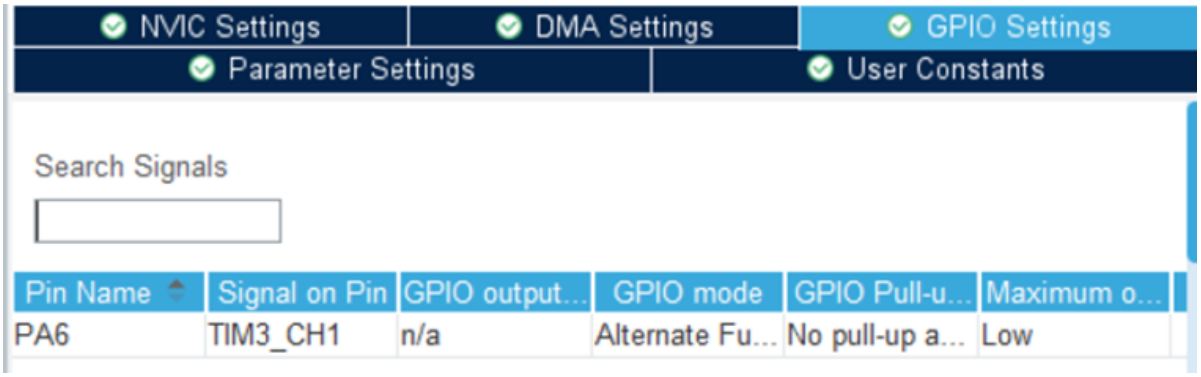
1. PWM配置

注:选择PWM模式1最好是把CH Polarity改为HIGH,这样CCRx越大PWM电平越大

- Clock Source: Internal Clock(选择内部时钟)
- Channel1:选择通道,一共四路,频率相同,对应四个GPIO,每个字都要对应哦
- PSC:记得小于65535
- ARR:CCRx<ARR则输出有效电平,CCRx>ARR则输出无效电平
- auto-reload : 自动重装,**记得打开使能**
- Mode:PWM1或者2
- CH Polarity:极性,建议**PWM1-HIGH**(这样CCRx越大,占空比越大)**PWM2-LOW**
- 不需要再NVIC使能中断,因为用不上



2. 记一下引脚,没什么事



5-2 基础代码

1. 前提

```
// TIM3--PWM的初始化,选择四个通道中的通道1(注意要先在Cube中选择使能相应通道channel)
HAL_TIM_PWM_Start(&htim3 , TIM_CHANNEL_1) ;
```

2. 核心代码

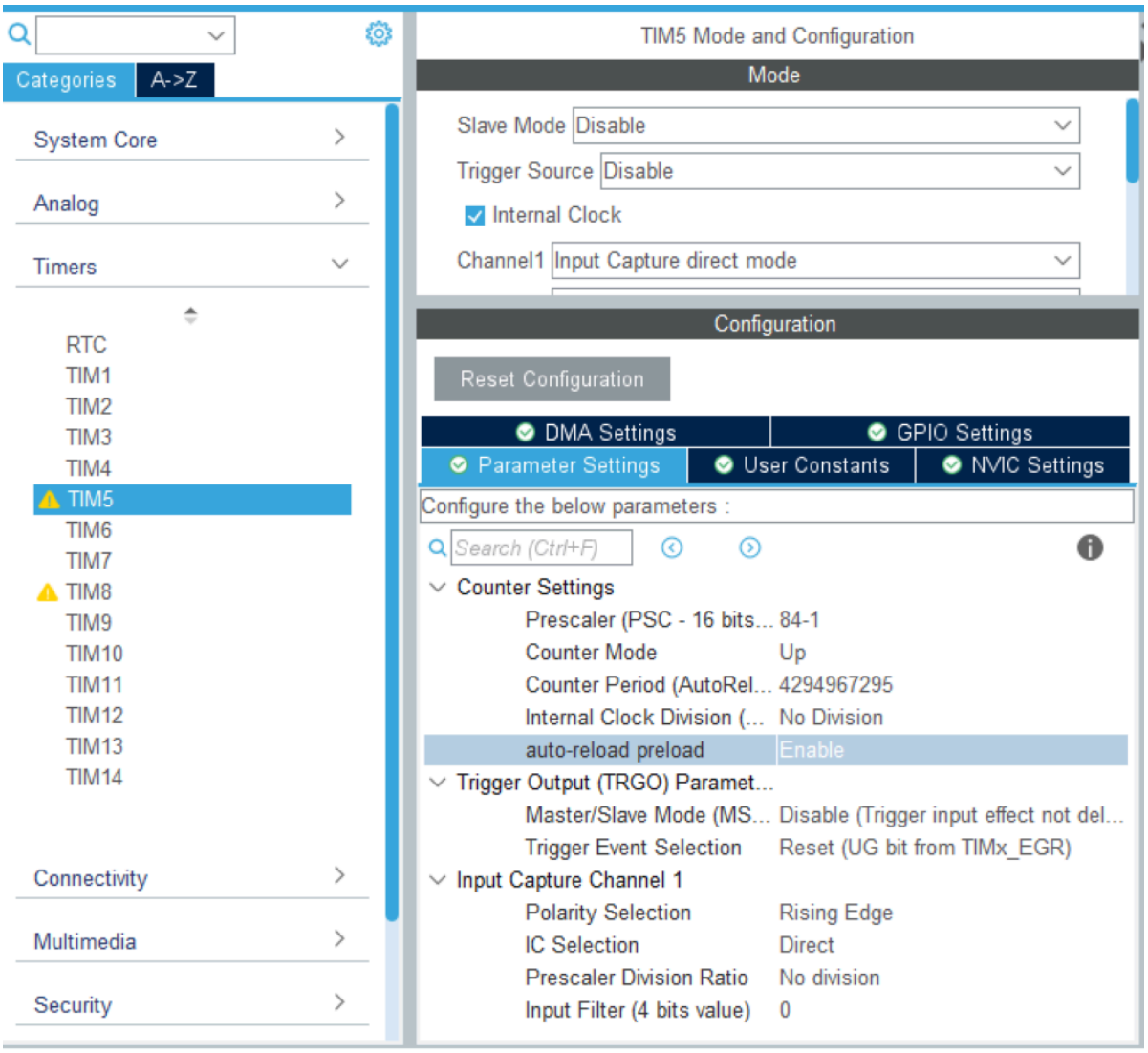
```
// 设置LED的占空比,LED_PWM需要在0-ARR之间
__HAL_TIM_SET_COMPARE(&htim3 , TIM_CHANNEL_1 , LED_PWM ) ;
```

6. TIM输入捕获

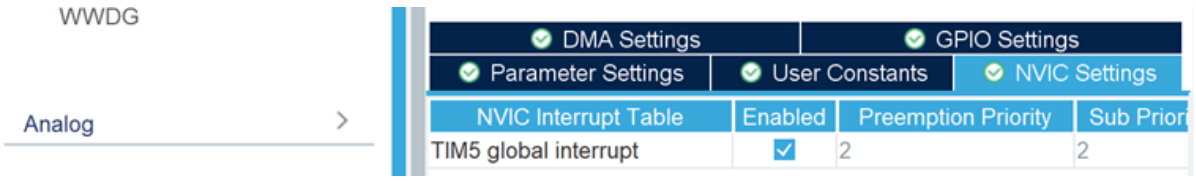
6-1 Cube配置(TIM5)

1. 参数配置

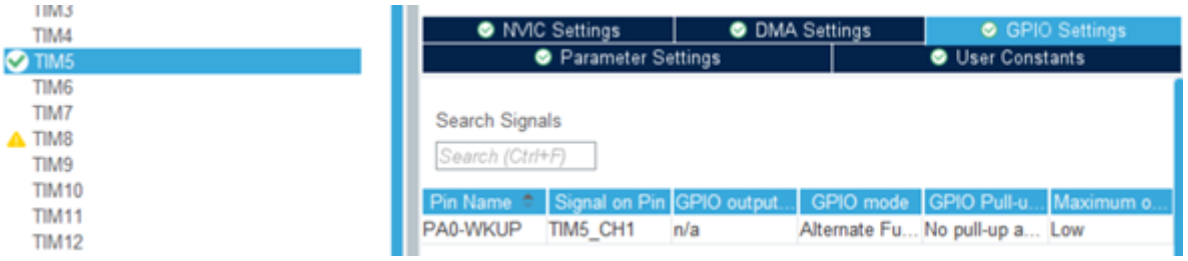
- 选择内部时钟
- channel1:输入捕获(IC)
- PSC没什么用,随便配置,给个84-1,PSC小.精度高,容易溢出,PSC大,精度低,可测量慢信号
- AAR设为最大,0xffffffff
- auto-reload preload(自动重装载):使能(Enable)
- 选择捕获边缘:比如Rising Edge表示捕获上升沿



2. 开启中断



3. 看看引脚



6-2 基础代码

1. setup初始化

```
// TIM5初始化
HAL_TIM_Base_Start_IT(&htim5);                // 启动更新中断
HAL_TIM_IC_Start_IT(&htim5, TIM_CHANNEL_1); // 启动输入捕获中断
```

2. 核心代码(捕获中断回调),以检测方波周期(上升沿到上升沿)为例

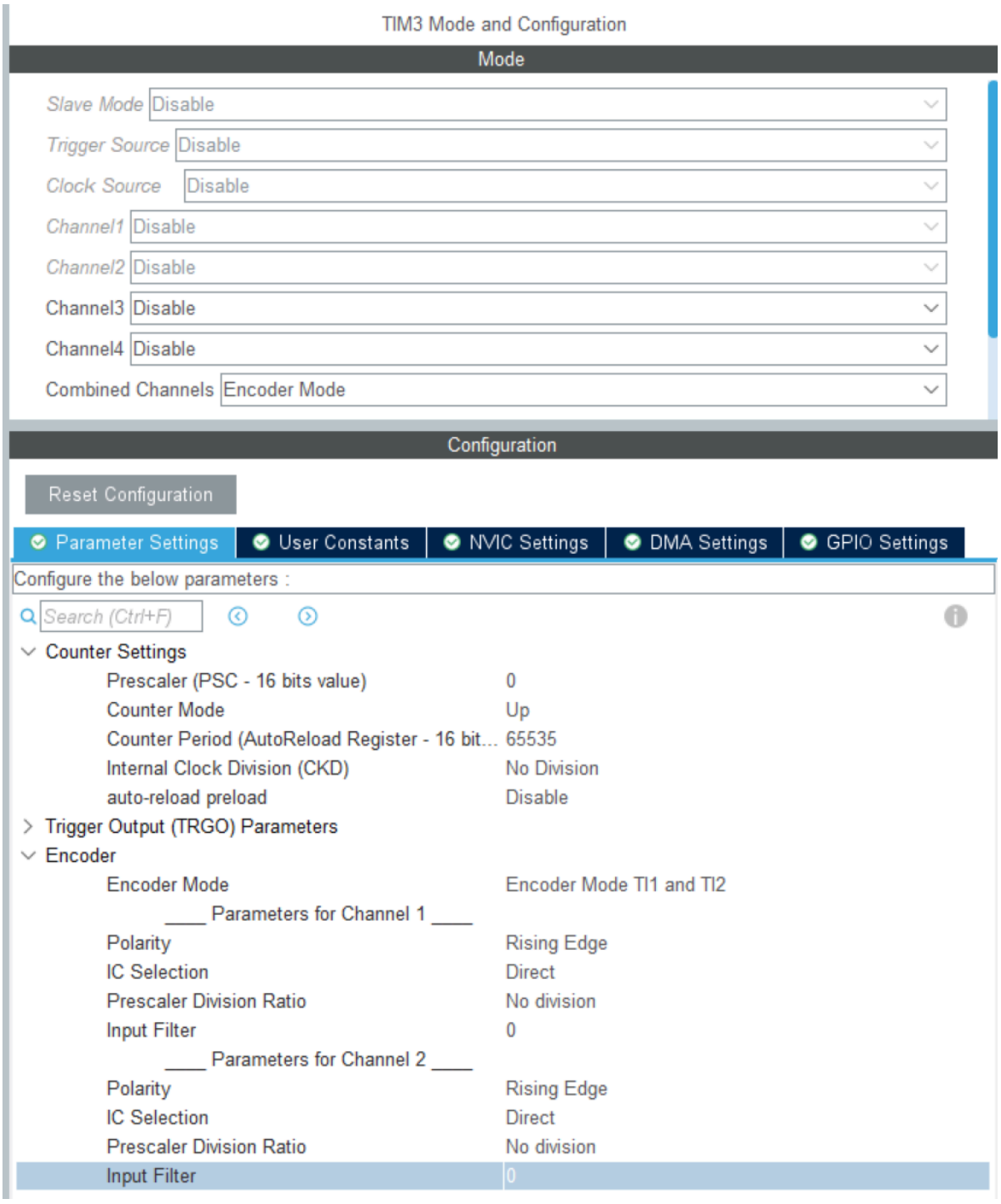
```c
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
        // 判断是否是TIM5且通道1引发的输入捕获中断
    if (htim->Instance == TIM5 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        if (IsFirstCapture == 0)
        {
            // 记录得到第一次上升沿时间,单位为tick
            IC_Val1 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
            IsFirstCapture = 1;
        }
        else
        {
            // 第二次上升沿
            IC_Val2 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);

            if (IC_Val2 > IC_Val1)
                CapturePeriod = IC_Val2 - IC_Val1;
            else
                CapturePeriod = (0xFFFFFFFF - IC_Val1) + IC_Val2 + 1;

            IsFirstCapture = 0;
        }
    }
}
```

7. TIM编码器模式

7-1 Cube配置

1. 参数配置

- 模式:Encoder Mode
- PSC = 0 , ARR = 65535(防止溢出)
- Encoder Mode : TI1 + TI2 (双边检测)
- Polarity:Rising(上升沿)
- Input Filter : 噪声滤波可以考虑,0-15为滤波标准



7-2 基础代码

1. setup初始化

```
// 启动编码器接口
HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
```

2. 中断中计算转速

```
void HAL_SYSTICK_Callback(void)
{
    // 启动自分频
    static int ms_count = 0 ;
    ms_count ++ ;

    // 功能1: 编码器测速
```

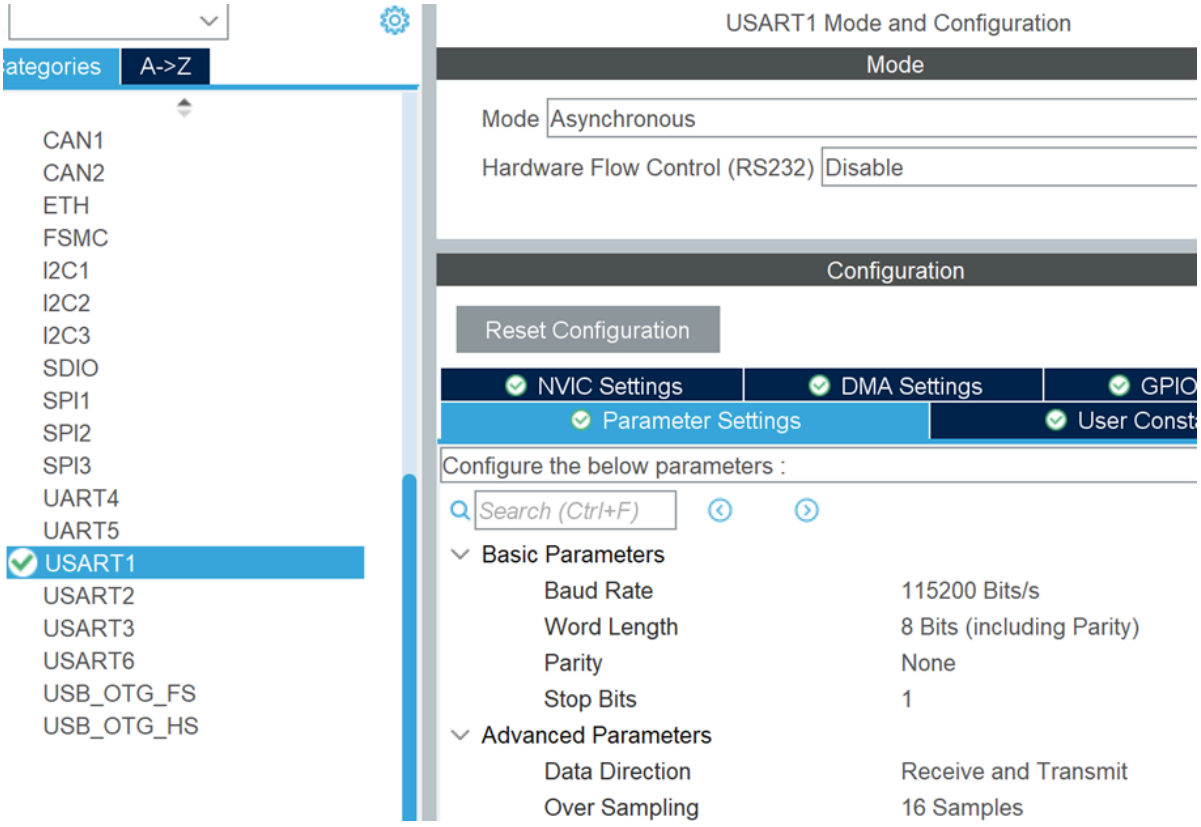
```
    if (ms_count == 1000)   // 采样间隔为1s
    {
        int16_t temp = __HAL_TIM_GET_COUNTER(&htim3) ;  // 得到一个采样周期间CNT的个数(不要溢出了否则CNT刷新,误差非常大)

        speed = temp / Cycle_CNT * 1 * 60 ;     // 1分钟的圈数(n/s) = 一个周期(1s)的CNT值 / 一圈对应的CNT值 * 时间

        __HAL_TIM_SET_COUNTER(&htim3 , 0) ; // 清零CNT,开启新的采样周期
        ms_count = 1 ;
    }
}
```

8. USART串口

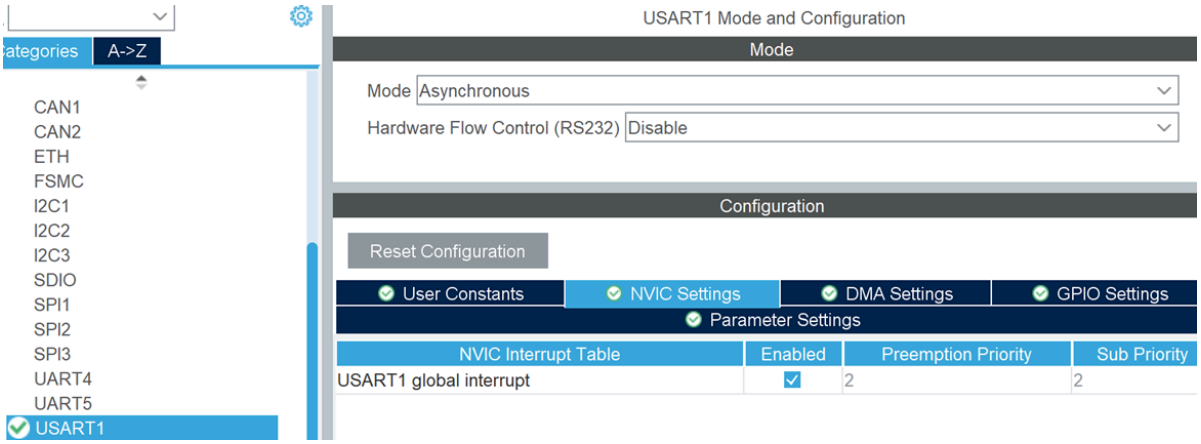
8-1 Cube配置

1. 参数配置

- Mode:Asynchronous(异步通信)
- 波特率:115200



2. 中断配置



8-2 基础代码

1. 全局变量

```
// 定义接受长度
#define USART_RX_BUF_LEN 25
// 接收状态和缓冲区
uint8_t USART_RX_BUF[USART_RX_BUF_LEN] ;
```

2. setup初始化

```
// 接收中断初始化
HAL_UARTEx_ReceiveToIdle_IT(&huart1, USART_RX_BUF, USART_RX_BUF_LEN);
```

3. 接收函数中断初始化

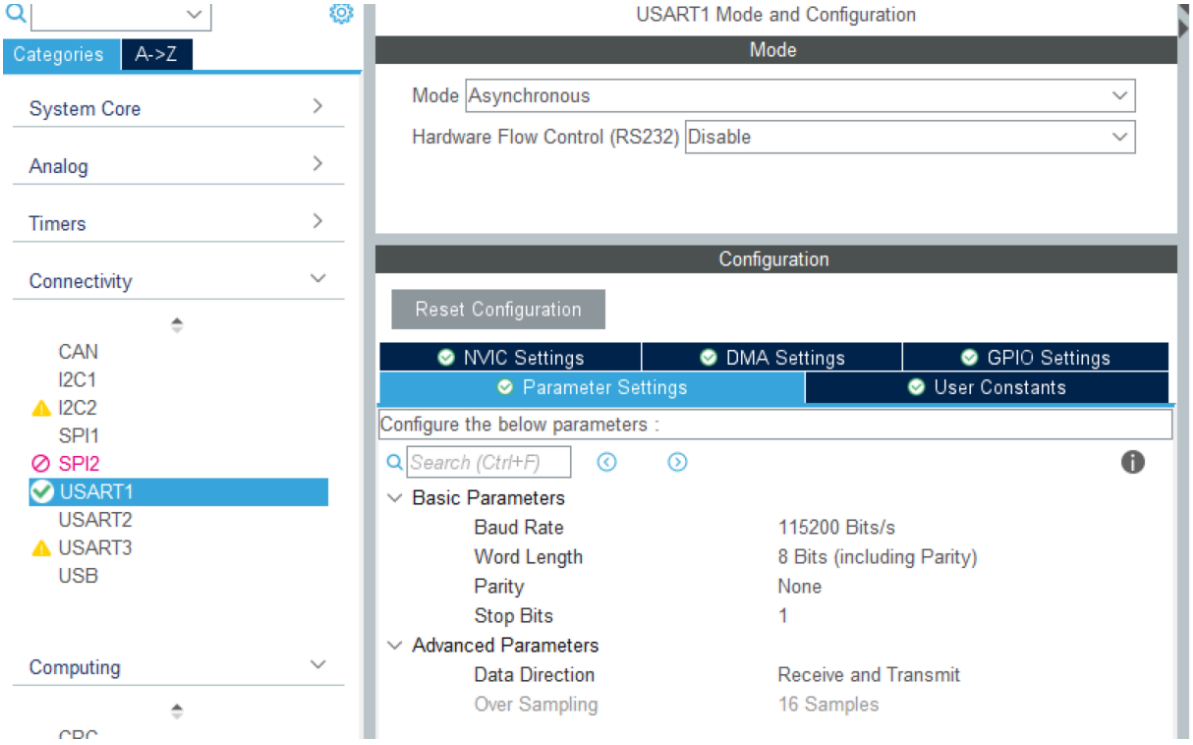
```
// USART串口通信
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
    if (huart->Instance == USART1)
    {
        // 1.添加结束符确保字符串安全
        if (Size > 0 && Size < USART_RX_BUF_LEN)
        {
            USART_RX_BUF[Size] = '\0';
        }

        // 2.使用字符串匹配代替固定长度检查
        if (strstr((char *)USART_RX_BUF, "send") != NULL)
        {
            HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);
        }

        // 3. 发送回显（添加\r\n+正确长度+状态检查）
        const char *echo_str = "Hello!"; // 不换行的回显字符串
        uint16_t echo_len = strlen(echo_str); // 计算长度

        if (huart1.gState == HAL_UART_STATE_READY)
        {
            HAL_UART_Transmit_IT(&huart1, (uint8_t *)echo_str, echo_len); // 检查UART是否就绪
        }
```

```
        // 4.重启接收前清除缓存区
        memset(USART_RX_BUF, 0, USART_RX_BUF_LEN);

        // 5.再次接收
        HAL_UARTEx_ReceiveToIdle_IT(&huart1, USART_RX_BUF, USART_RX_BUF_LEN);
    }
}
```

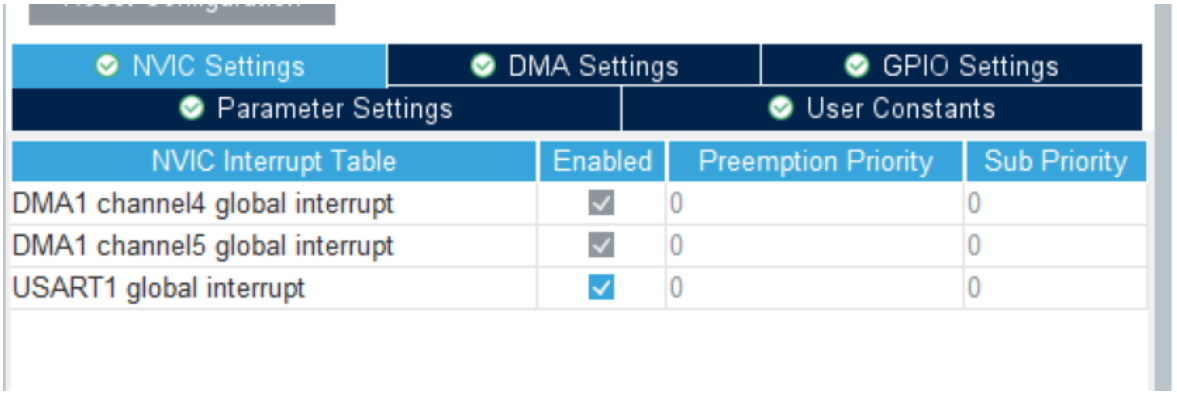
9. USART_DMA

9-1 Cube配置

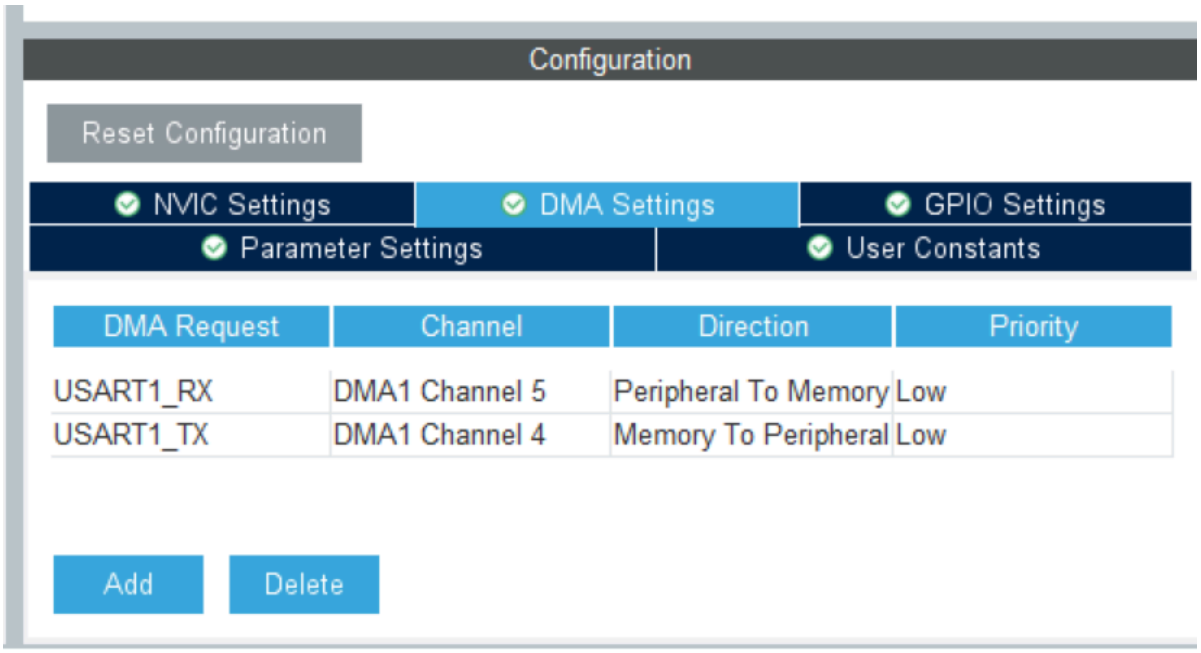
- 基础配置



- NVIC配置



- DMA直接Add两个即可



9-2 基础代码

- 全局变量

```
// DMA串口接收变量
#define RX_USART1_LEN 50
uint8_t RX_USART1[RX_USART1_LEN] ;  // 接收数组
```

- setup初始化

```
HAL_UARTEx_ReceiveToIdle_DMA(&huart1 , RX_USART1 , RX_USART1_LEN ) ;// 开启DMA+接收空闲中断,接收的数据都存储在RX_USART1
```

- DMA发送数据

```
uint8_t sendbuff[] = "hello" ;
HAL_UART_Transmit_DMA(&huart1, (uint8_t *)sendbuff, sizeof(sendbuff));
```

- DMA接收中断

```
// 串口空闲中断回调函数
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
    if (huart->Instance == USART1)
    {
        // 加上\0防止越界
        if (Size < RX_USART1_LEN)
        {
            RX_USART1[Size] = '\0';
        }

        // 清理缓冲区剩余数据，防止旧数据残留
    memset(RX_USART1 + Size, 0, RX_USART1_LEN - Size);

        // 发送回显
```

```
        HAL_UART_Transmit_DMA(&huart1, (uint8_t *)RX_USART1, sizeof(RX_USART1));

        // 每次处理完需要重新开启DMA中断
        HAL_UARTEx_ReceiveToIdle_DMA(&huart1 , RX_USART1 , RX_USART1_LEN ) ;
    }
}
```